

Advanced Deep Learning

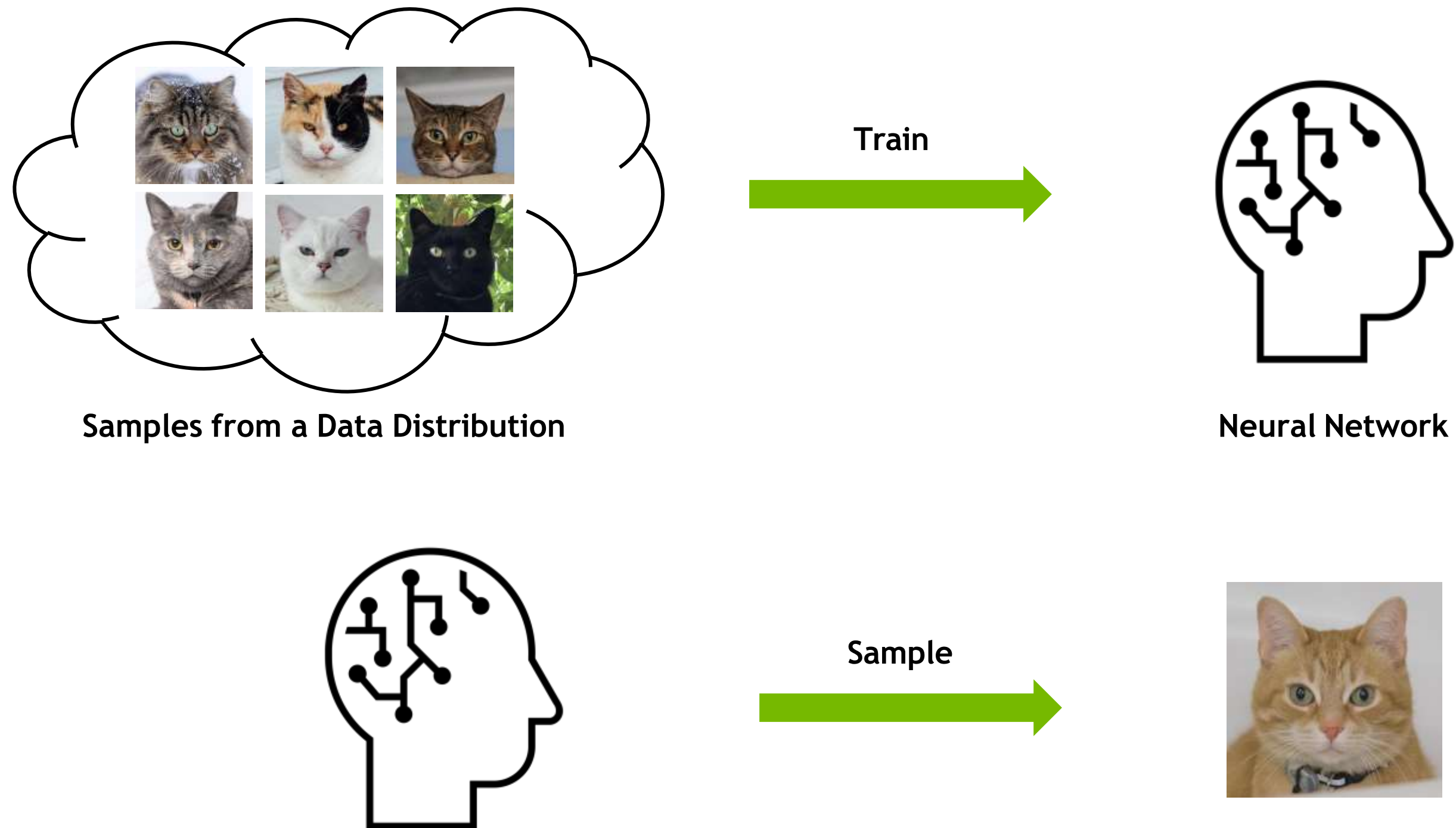
Diffusion Model

Prof. Jianguo Zhang
SUSTech

Slides were adapted from various sources

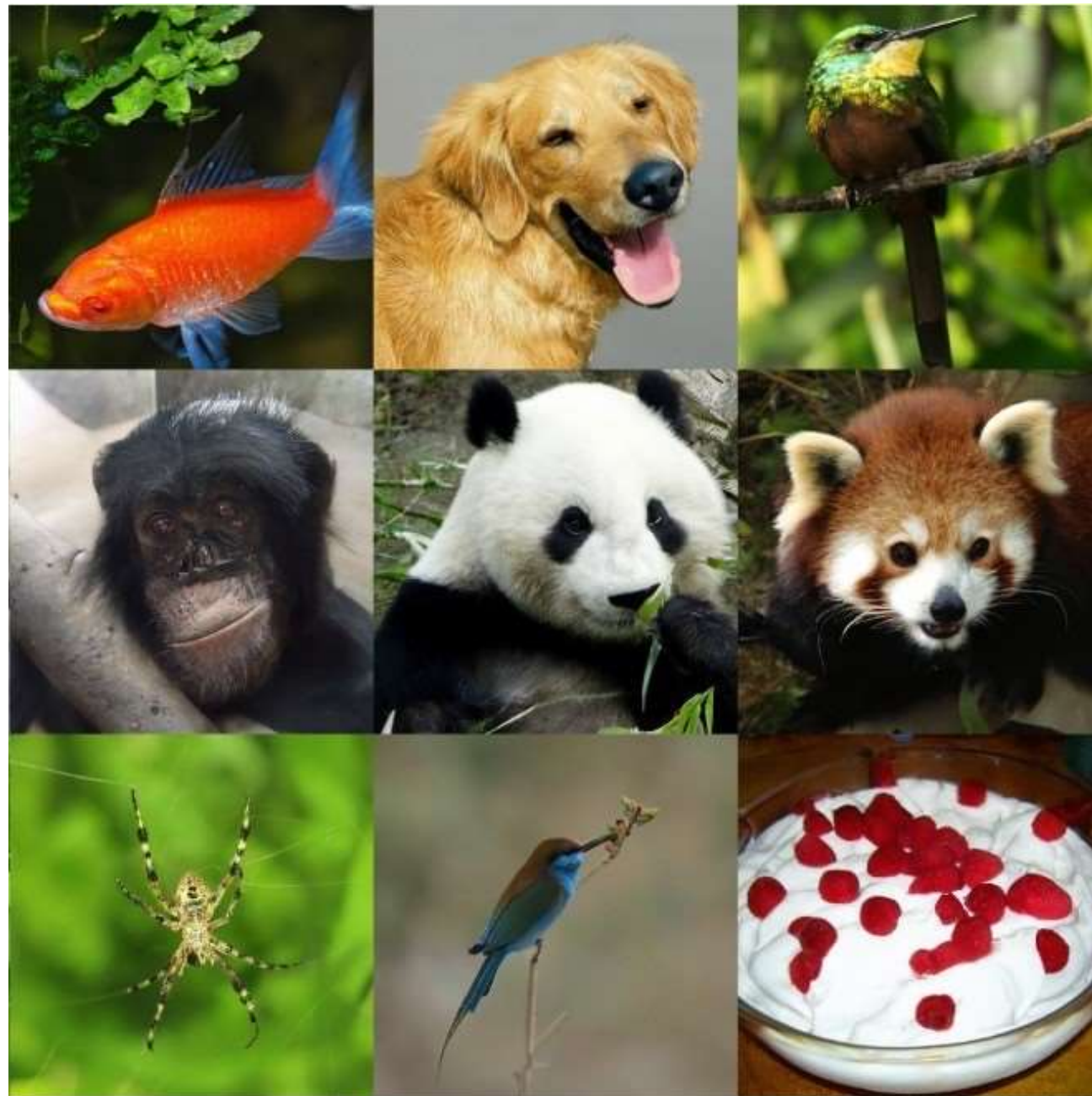
Deep Generative Learning

Learning to generate data



Denoising Diffusion Models

Emerging as powerful generative models, outperforming GANs



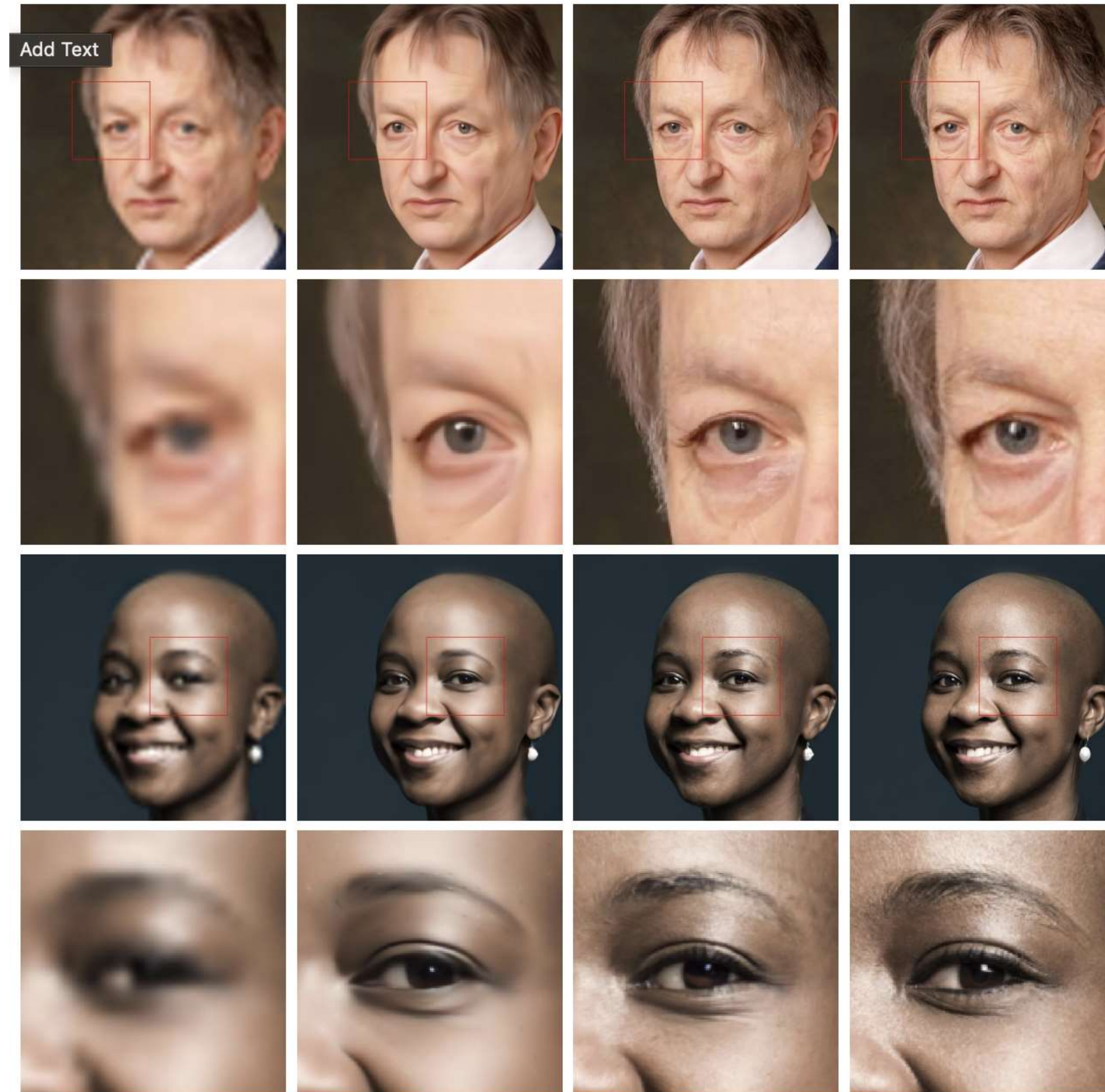
[“Diffusion Models Beat GANs on Image Synthesis”](#)
Dhariwal & Nichol, OpenAI, 2021



[“Cascaded Diffusion Models for High Fidelity Image Generation”](#)
Ho et al., Google, 2021

Image Super-resolution

Successful applications



Text-to-Image Generation

DALL·E 2

“a teddy bear on a skateboard in times square”



[“Hierarchical Text-Conditional Image Generation with CLIP Latents”](#)
Ramesh et al., 2022

Imagen

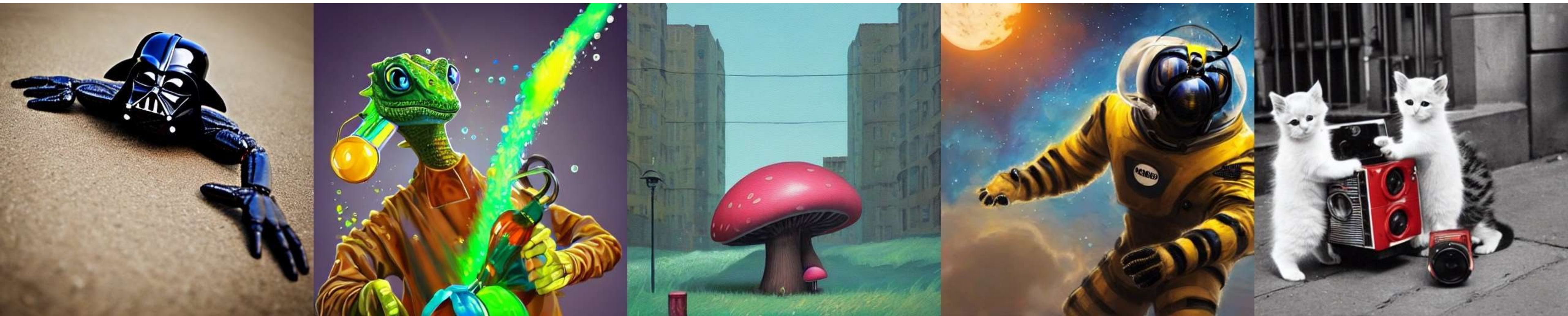
A group of teddy bears in suit in a corporate office celebrating the birthday of their friend. There is a pizza cake on the desk.



[“Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”](#), Saharia et al., 2022

Text-to-Image Generation

Stable Diffusion



[Stable Diffusion Applications: Twitter Mega Thread](#)

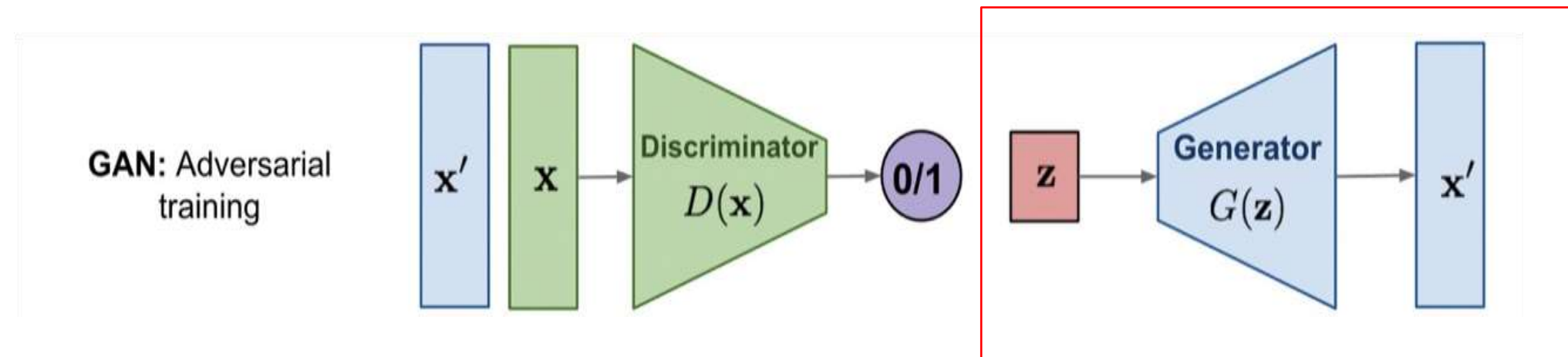
[“High-Resolution Image Synthesis with Latent Diffusion Models”](#) [Rombach et al., 2022](#)

Q: What is a diffusion model?



GANs

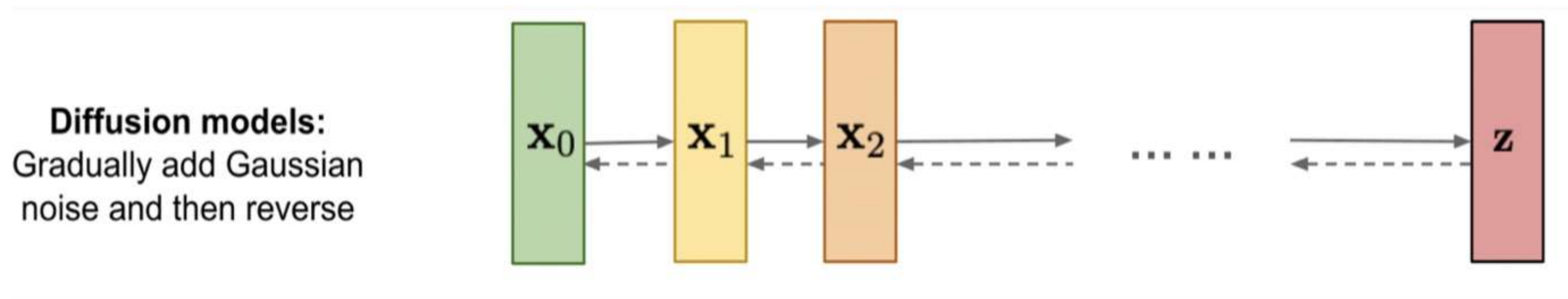
- Convert latent noise vector to target distribution in one step



- Lots of issues with training:
 - Vanishing gradients: if discriminator is too good, gradients go to zero
 - Mode collapse: if the generator learns to generate an exceptionally plausible output, it will just continue generating it. Then the discriminator will learn to always reject it, and then the generator will produce the same outputs, which the discriminator will then reject... bad loop!

Diffusion

- Idea: Estimating and analyzing small step sizes is more tractable/easier than a single step from random noise to the learned distribution
- Convert a well-known and simple *base distribution* (like a Gaussian) to the *target (data) distribution* iteratively, with small step sizes, via a Markov chain:



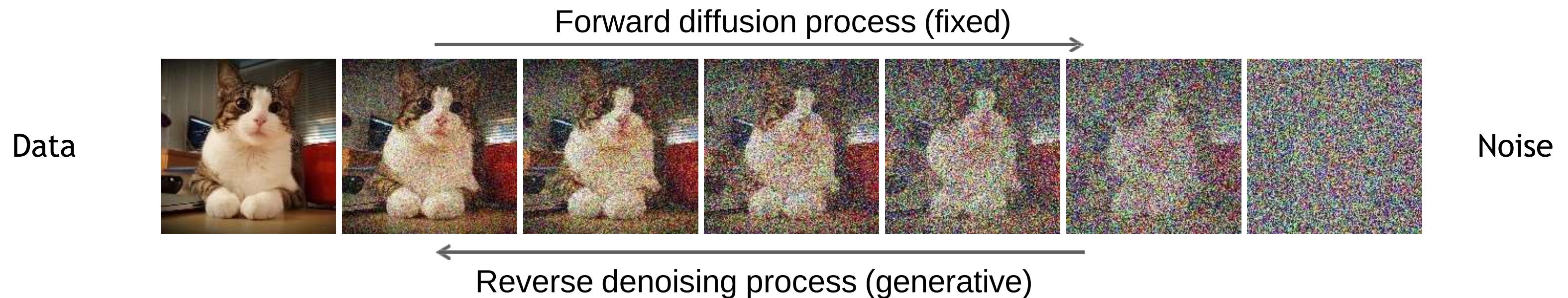
- Markov chain: outlines the probability associated with a sequence of events occurring based on the state in the previous event.

Denoising Diffusion Models

Learning to generate by denoising

Denoising diffusion models consist of two processes:

- Forward diffusion process that gradually adds noise to input
- Reverse denoising process that learns to generate data by denoising



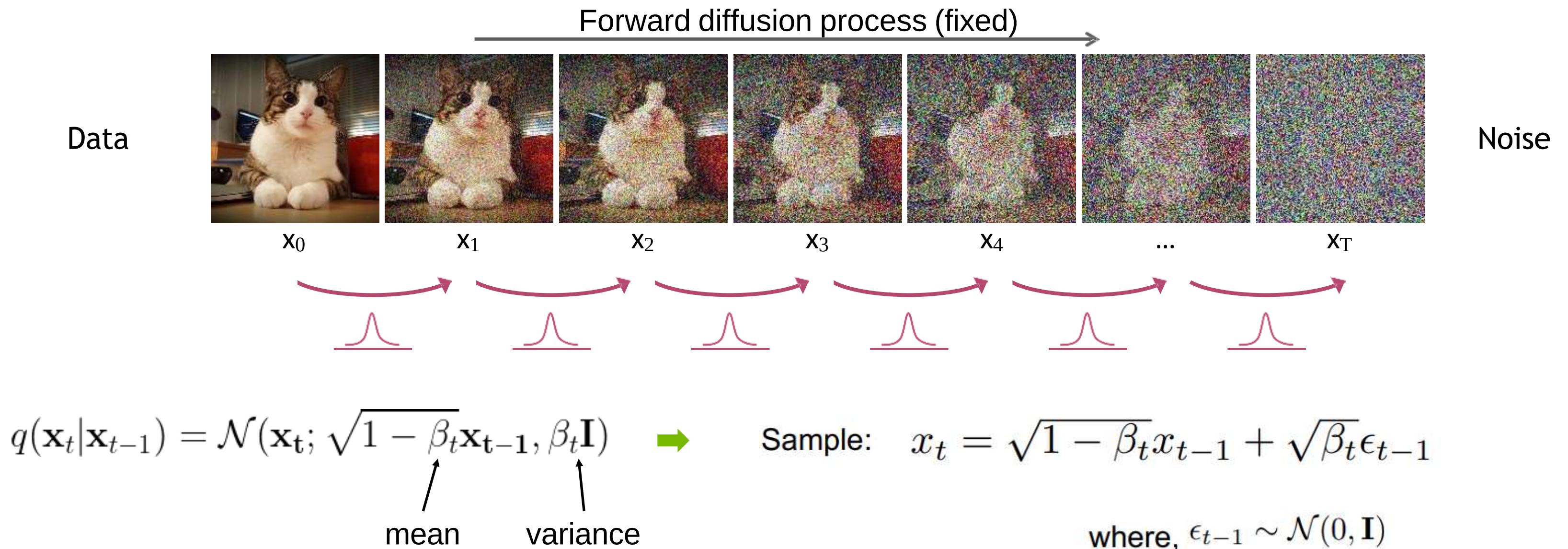
[Sohl-Dickstein et al., Deep Unsupervised Learning using Nonequilibrium Thermodynamics, ICML 2015](#)

[Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020](#)

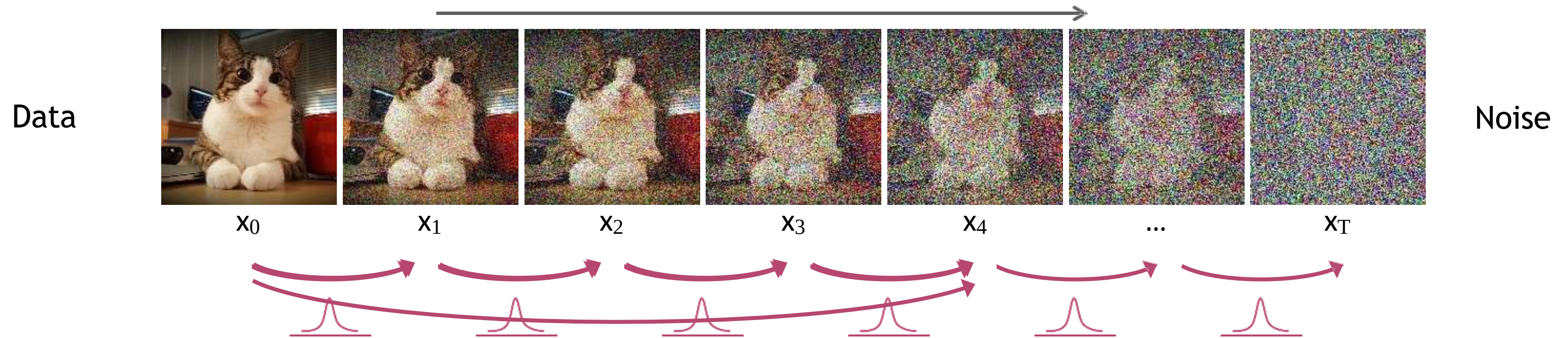
[Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021](#)

Forward Diffusion Process

The formal definition of the forward process in T steps:



Forward Diffusion Process



$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \underbrace{\sqrt{1 - \beta_t} \mathbf{x}_{t-1}}_{\text{mean}}, \underbrace{\beta_t \mathbf{I}}_{\text{variance}}) \quad \rightarrow \quad \text{Sample: } x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon_{t-1}$$

where, $\epsilon_{t-1} \sim \mathcal{N}(0, \mathbf{I})$

$$\text{Define, } \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s) \quad \rightarrow \quad q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

For sampling: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$ where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

β_t values schedule (i.e., the noise schedule) is designed and: $\bar{\alpha}_T \rightarrow 0$

$$q(x_T | x_0) \approx q(x_T) \approx \mathcal{N}(x_T; 0, I)$$

What happens to a distribution in the forward diffusion?

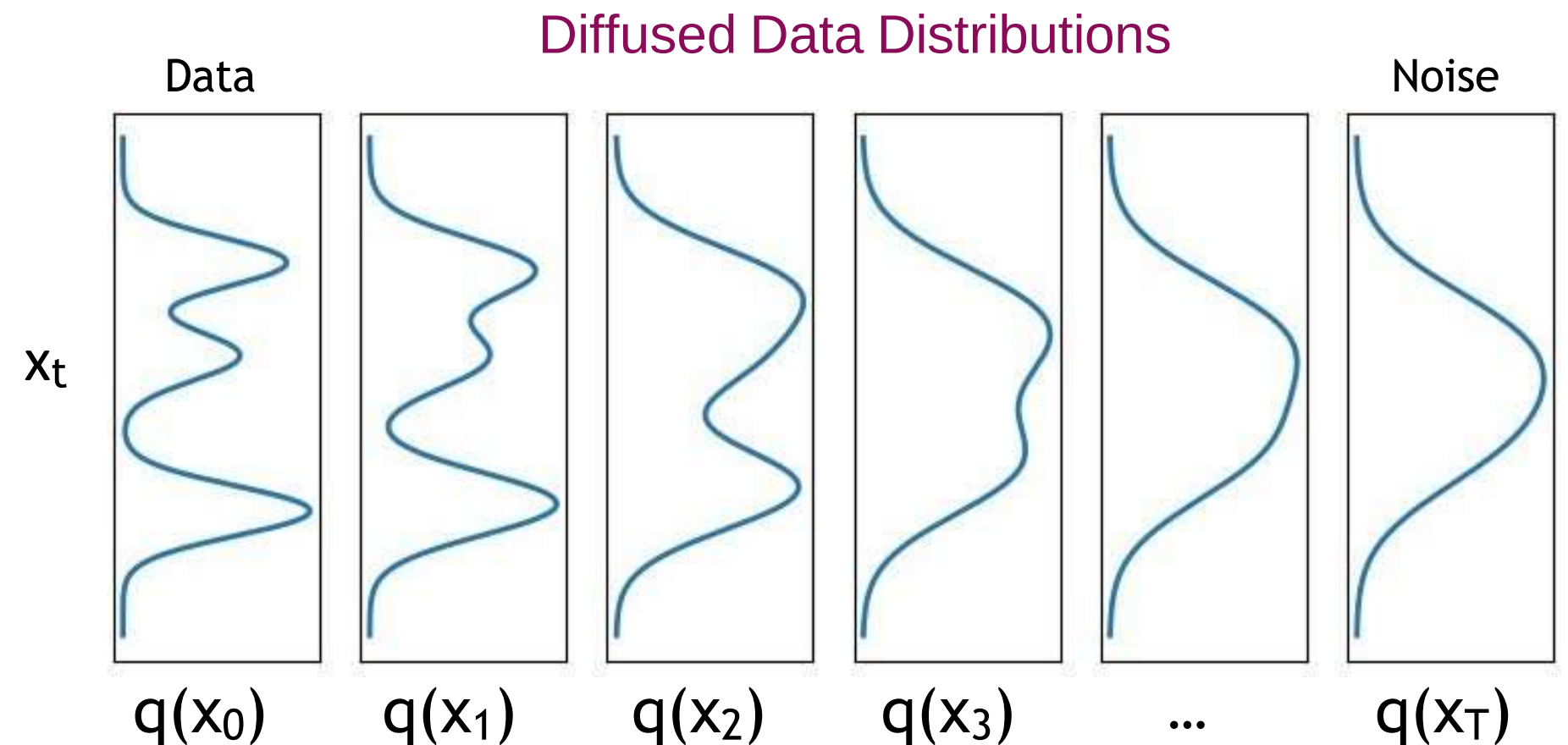
$$\underbrace{q(\mathbf{x}_t)}_{\text{Diffused data dist.}} = \int \underbrace{q(\mathbf{x}_0, \mathbf{x}_t)}_{\text{Joint dist.}} d\mathbf{x}_0 = \int \underbrace{q(\mathbf{x}_0)}_{\text{Input data dist.}} \underbrace{q(\mathbf{x}_t|\mathbf{x}_0)}_{\text{Diffusion Forward dist.}} d\mathbf{x}_0$$

We can sample
by first sampling
and then sampling

$$\mathbf{x}_t \sim q(\mathbf{x}_t)$$

$$\mathbf{x}_0 \sim q(\mathbf{x}_0) \text{ (Clean image)}$$

$$\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_0) \quad \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon$$



Generative Learning by Denoising

Recall, that the diffusion parameters are designed such that $q(\mathbf{x}_T) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$ (Pure Noise)

Diffused Data Distributions

Generation:

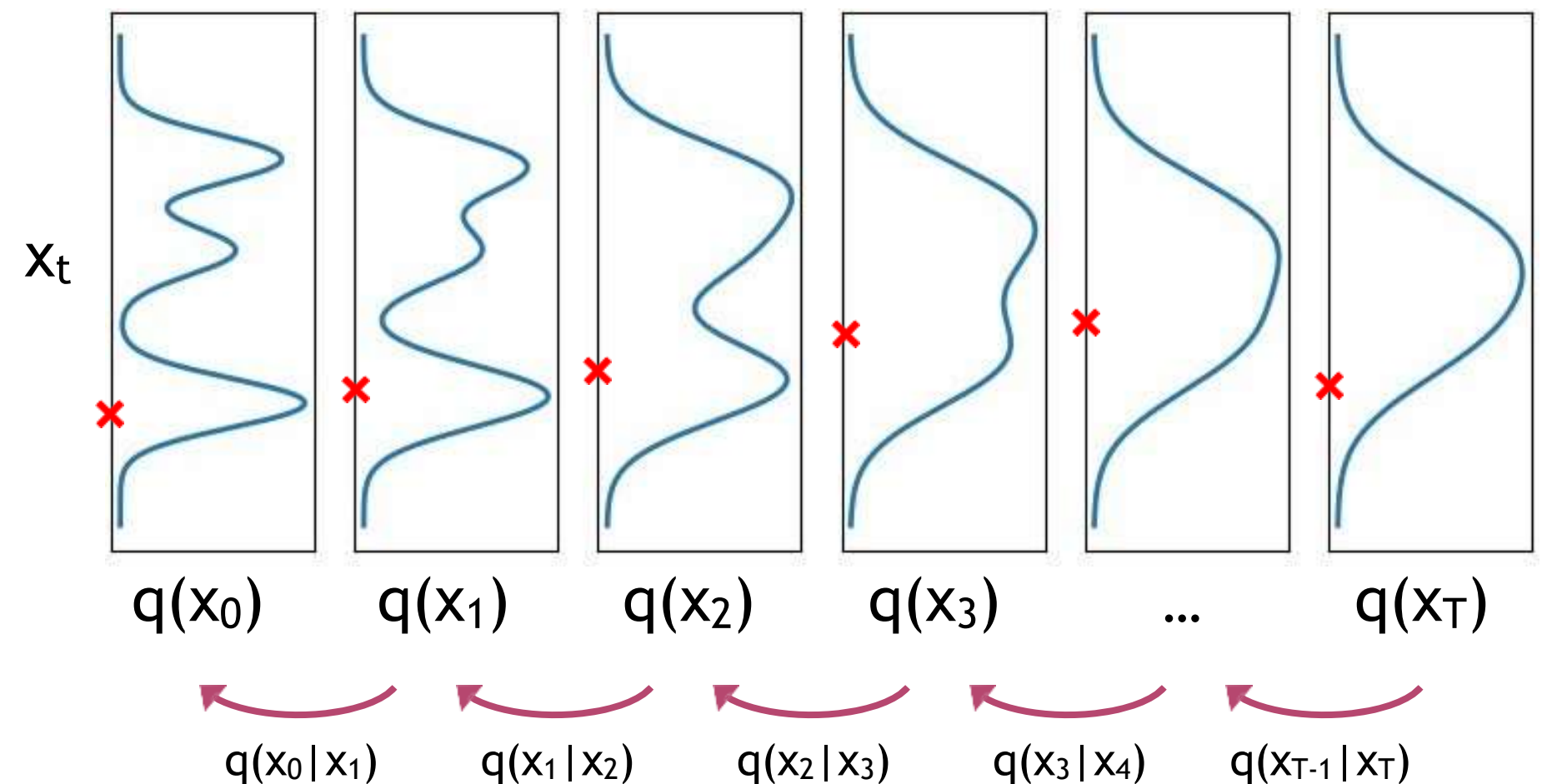
Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

Iteratively sample $\mathbf{x}_{t-1} \sim \underbrace{q(\mathbf{x}_{t-1}|\mathbf{x}_t)}_{\text{True Denoising Dist.}}$

$q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is unknown

we can use a **Normal distribution** if:

β_t is small in each forward diffusion step.



Generative Learning by Denoising

For small enough forward steps, i.e. $\{\beta_t \in (0, 1)\}_{t=1}^T$

the reverse process step $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ can be estimated as a Gaussian distribution

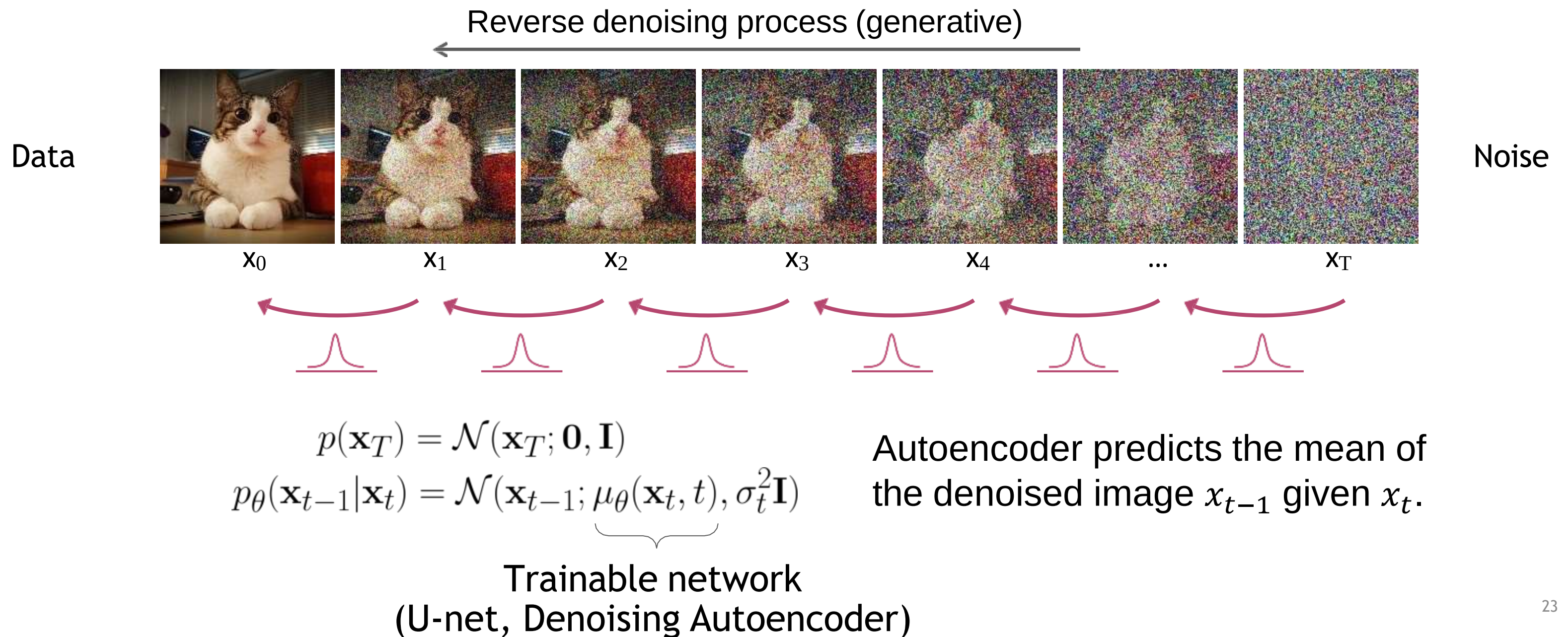
Therefore, we can parametrize the *learned* reverse process as

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \boldsymbol{\Sigma}_{\theta}(\mathbf{x}_t, t))$$

In practice, Σ is set to $\sigma_t^2 I$ (σ_t as hyperparameter), so we only need to learn the mean of the distribution

Training a Diffusion Model

Learning to denoise: learning $\mu_\theta(x_t, t)$



Training a Diffusion Model

Learning to denoise: learning $\mu_\theta(x_t, t)$ Trainable network
(U-net, Denoising Autoencoder)

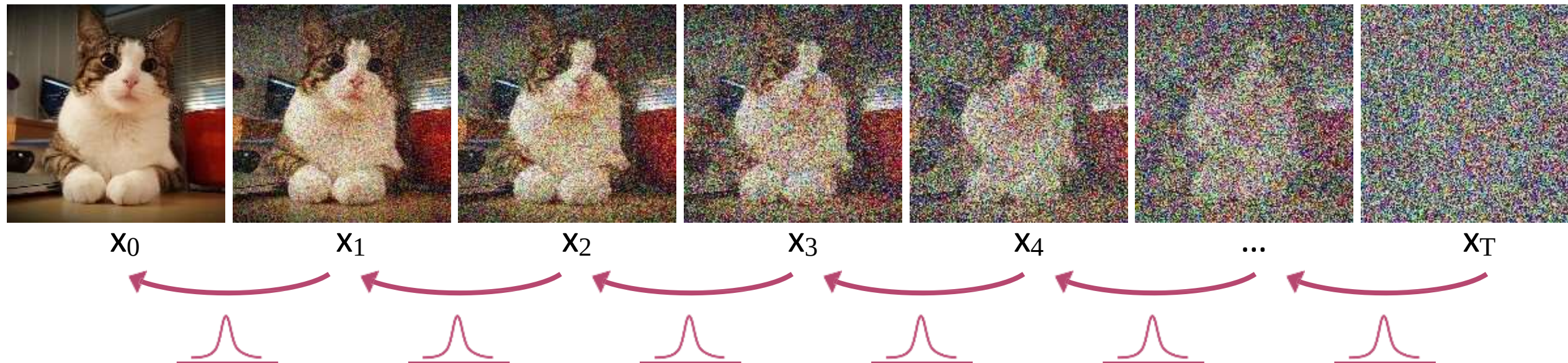
Ground Truth (Posterior Distribution):

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}),$$

where $\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t$ and $\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$

Reverse denoising process (generative)

Data



Training a Diffusion Model

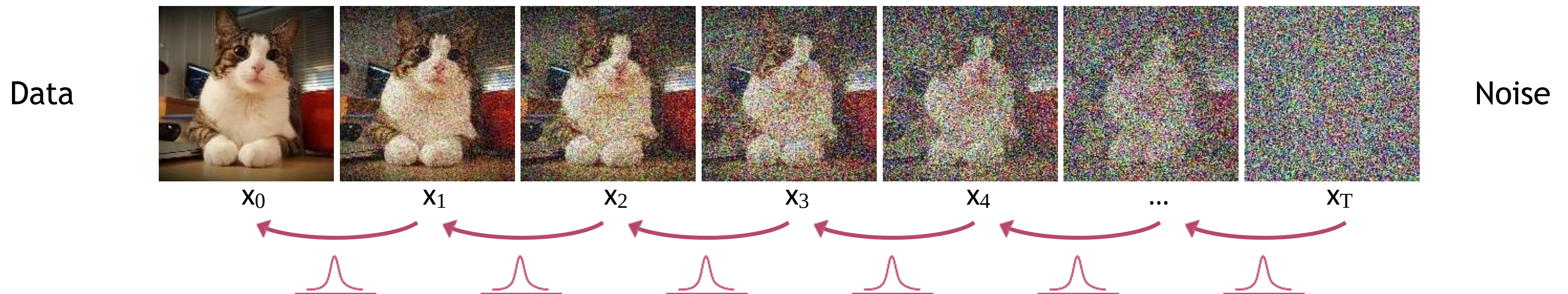
Learning to denoise: learning $p_\theta(x_{t-1}|x_t)$: $\mathcal{N}(\mathbf{x}_{t-1}; \underline{\mu_\theta(\mathbf{x}_t, t)}, \sigma_t^2 \mathbf{I})$

Ground truth (Posterior Distribution): $q(x_{t-1}|x_t, x_0)$: $\mathcal{N}(\mathbf{x}_{t-1}; \underline{\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0)}, \tilde{\beta}_t \mathbf{I})$

Loss function: KL divergence

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$

Reverse denoising process (generative)



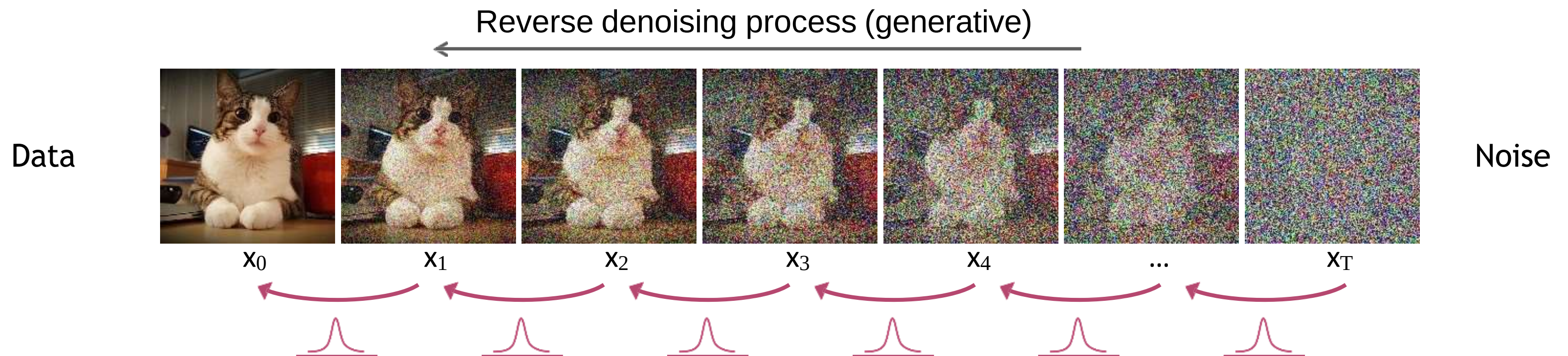
Training a Diffusion Model

Learning to denoise: learning $p_{\theta}(x_{t-1}|x_t)$: $\mathcal{N}(\mathbf{x}_{t-1}; \underline{\mu_{\theta}(\mathbf{x}_t, t)}, \sigma_t^2 \mathbf{I})$

Ground truth (Posterior Distribution): $q(x_{t-1}|x_t, x_0)$: $\mathcal{N}(\mathbf{x}_{t-1}; \underline{\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0)}, \tilde{\beta}_t \mathbf{I})$

Loss function: KL divergence or Simply align μ_{θ} & $\tilde{\mu}_t$ with MSE

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_{\theta}(\mathbf{x}_t, t)\|^2 \right] + C$$



Training a Diffusion Model

Loss function: align μ_θ & $\tilde{\mu}_t$ with MSE

With further parameterization

Recall that $\mathbf{x}_t(\mathbf{x}_0, \epsilon) = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t = \frac{1}{\sqrt{\alpha_t}} \left(\overbrace{\mathbf{x}_t(\mathbf{x}_0, \epsilon)}^{\text{Input to model}} - \underbrace{\frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon}_{\text{Added noise}} \right)$$

Why not just predict the residual (the added noise)?

$$\mu_\theta(\mathbf{x}_t, t) = \tilde{\mu}_t \left(\mathbf{x}_t, \underbrace{\frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_\theta(\mathbf{x}_t))}_{\text{Estimated } x_0 \text{ given predicted noise } \epsilon_\theta} \right) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon_\theta(\mathbf{x}_t, t) \right)$$

Estimated x_0 given
predicted noise ϵ_θ

The denoising Net

How do we train? (summary version)

Final loss function with noise prediction ([Ho et al. NeurIPS 2020](#))

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1, T)} \left[\left\| \epsilon - \underbrace{\epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)}_{\mathbf{x}_t} \right\|^2 \right]$$

Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on
 $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$
 - 6: **until** converged
-

U-Net autoencoder takes $\mathbf{x}(t)$ as input and simply predict a noise.
The goal of the training is to generate a noise pattern that is unit normal.

Summary

Training and Sample Generation

Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$   
6: until converged
```

Algorithm 2 Sampling

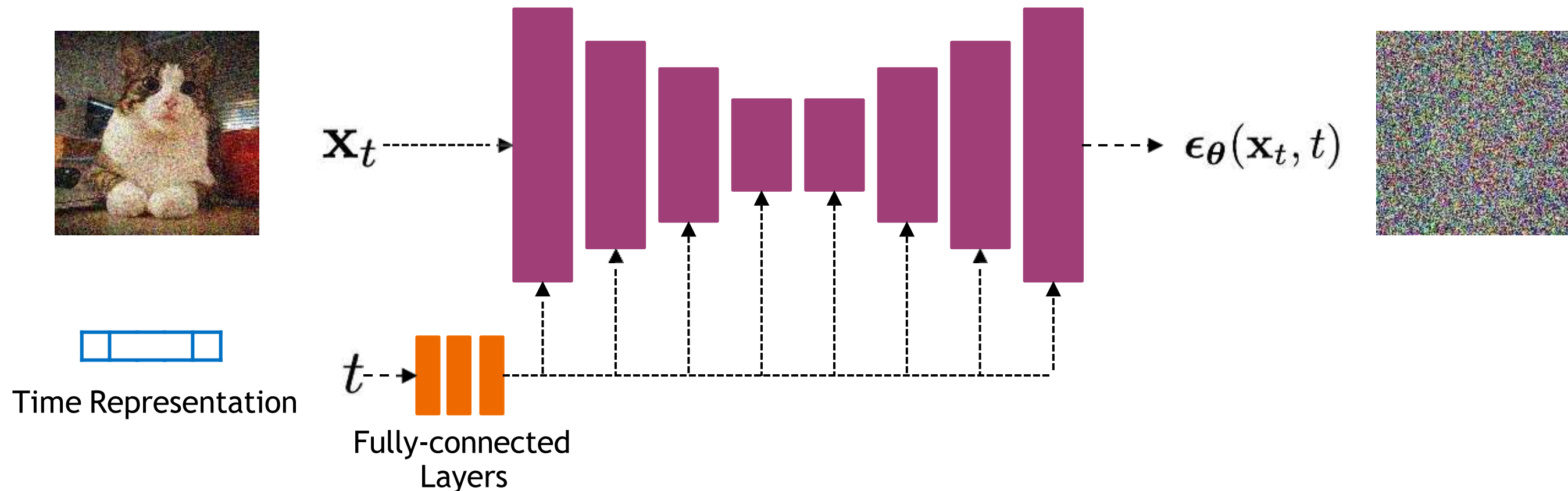
```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

Intuitively: During forward process we add noise to image. During reverse process we try to predict that noise with a U-Net and then subtract it from the image to denoise it.

Implementation Considerations

Network Architectures

Diffusion models often use U-Net architectures with ResNet blocks and self-attention layers to represent $\epsilon_{\theta}(\mathbf{x}_t, t)$



Time representation: sinusoidal positional embeddings or random Fourier features.

Time features are fed to the residual blocks using either simple spatial addition or using adaptive group normalization layers. (see [Dharivwal and Nichol NeurIPS 2021](#))

Diffusion Parameters

Noise Schedule

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

Data



Noise

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2\mathbf{I})$$

Above, β_t and σ_t^2 control the variance of the forward diffusion and reverse denoising processes respectively.

Often a linear schedule is used for β_t , and σ_t^2 is set equal to β_t . Slowly increase the amount of added noise.

[Kingma et al. NeurIPS 2022](#) introduce a new parameterization of diffusion models using signal-to-noise ratio (SNR), and show how to learn the noise schedule by minimizing the variance of the training objective.

We can also train σ_t^2 while training the diffusion model by minimizing the variational bound ([Improved DPM by Nichol and Dhariwal ICML 2021](#)) or after training the diffusion model ([Analytic-DPM by Bao et al. ICLR 2022](#)).

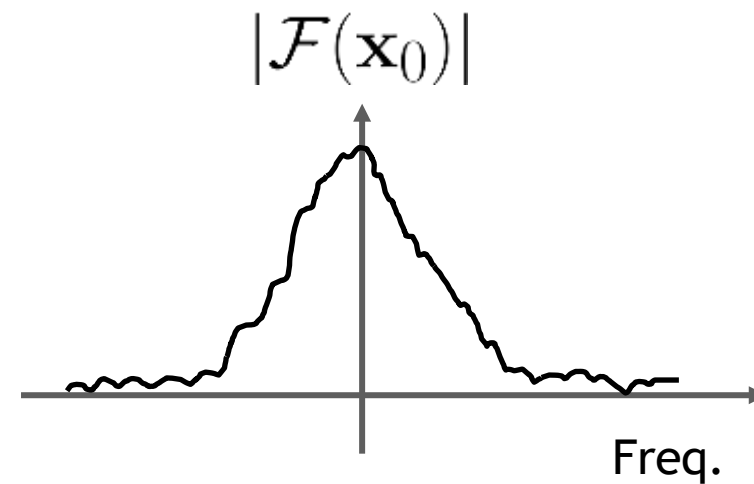
What happens to an image in the forward diffusion process?

Recall that sampling from $q(\mathbf{x}_t|\mathbf{x}_0)$ is done using $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$ where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

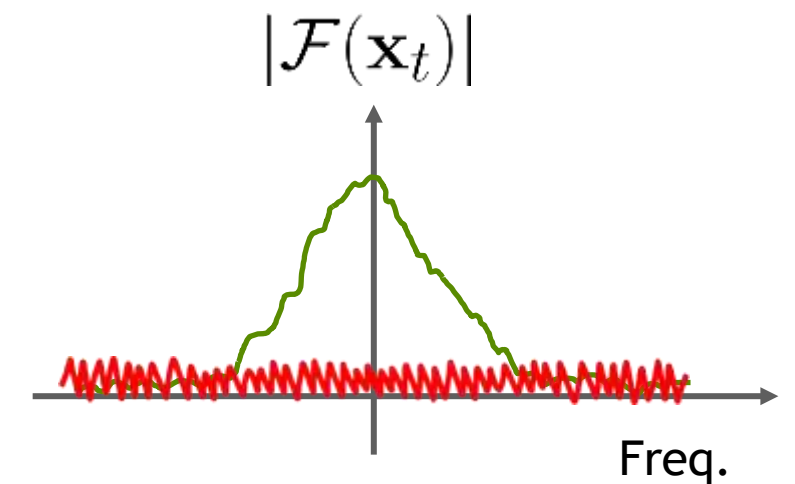
$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$$

Fourier Transform

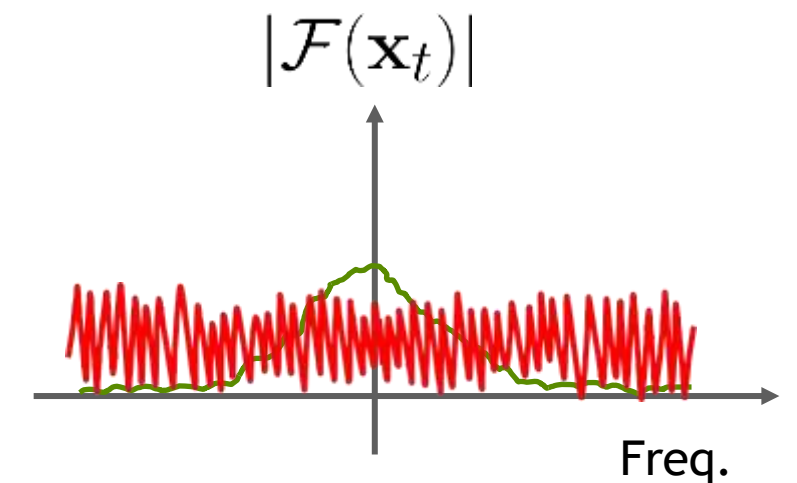
$$\mathcal{F}(\mathbf{x}_t) = \sqrt{\bar{\alpha}_t} \mathcal{F}(\mathbf{x}_0) + \sqrt{(1 - \bar{\alpha}_t)} \mathcal{F}(\epsilon)$$



Small t
 $\bar{\alpha}_t \sim 1$



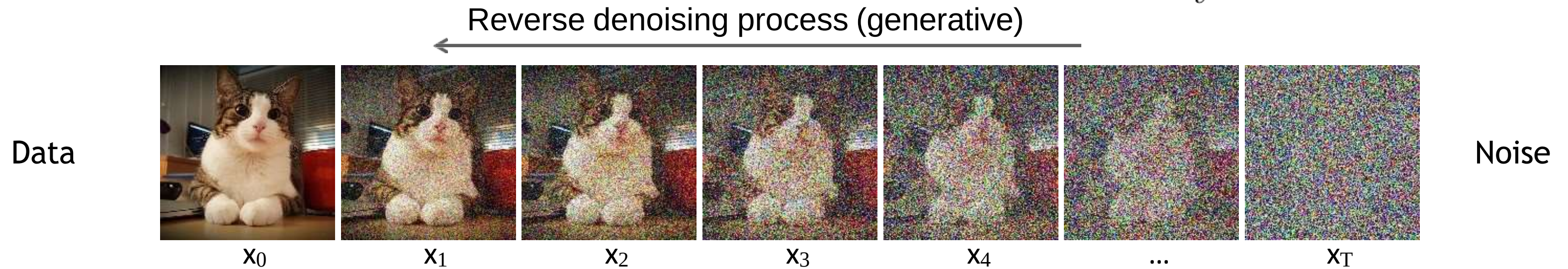
Large t
 $\bar{\alpha}_t \sim 0$



In the forward diffusion, the high frequency content is perturbed faster.

Content-Detail Tradeoff

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1, T)} \left[\left\| \epsilon - \underbrace{\epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)}_{\mathbf{x}_t} \right\|^2 \right]$$



The denoising model is specialized for generating the high-frequency content (i.e., low-level details)

The denoising model is specialized for generating the low-frequency content (i.e., coarse content)

The weighting of the training objective for different timesteps is important!

Summary

Denoising Diffusion Probabilistic Models

- Diffusion process can be reversed if the variance of the gaussian noise added at each step of the diffusion is small enough.
- To reverse the process we train a U-Net that takes input: current noisy image and timestamp, and predicts the noise map..
- Training goal is to make sure that the predicted noise map at each step is unit gaussian (Note that in VAE we also required the latent space to be unit gaussian).
- During sampling/generation, subtract the predicted noise from the noisy image at time t to generate the image at time $t - 1$ (with some weighting).

Q: How to do conditional generation?



Impressive conditional diffusion models

Text-to-image generation

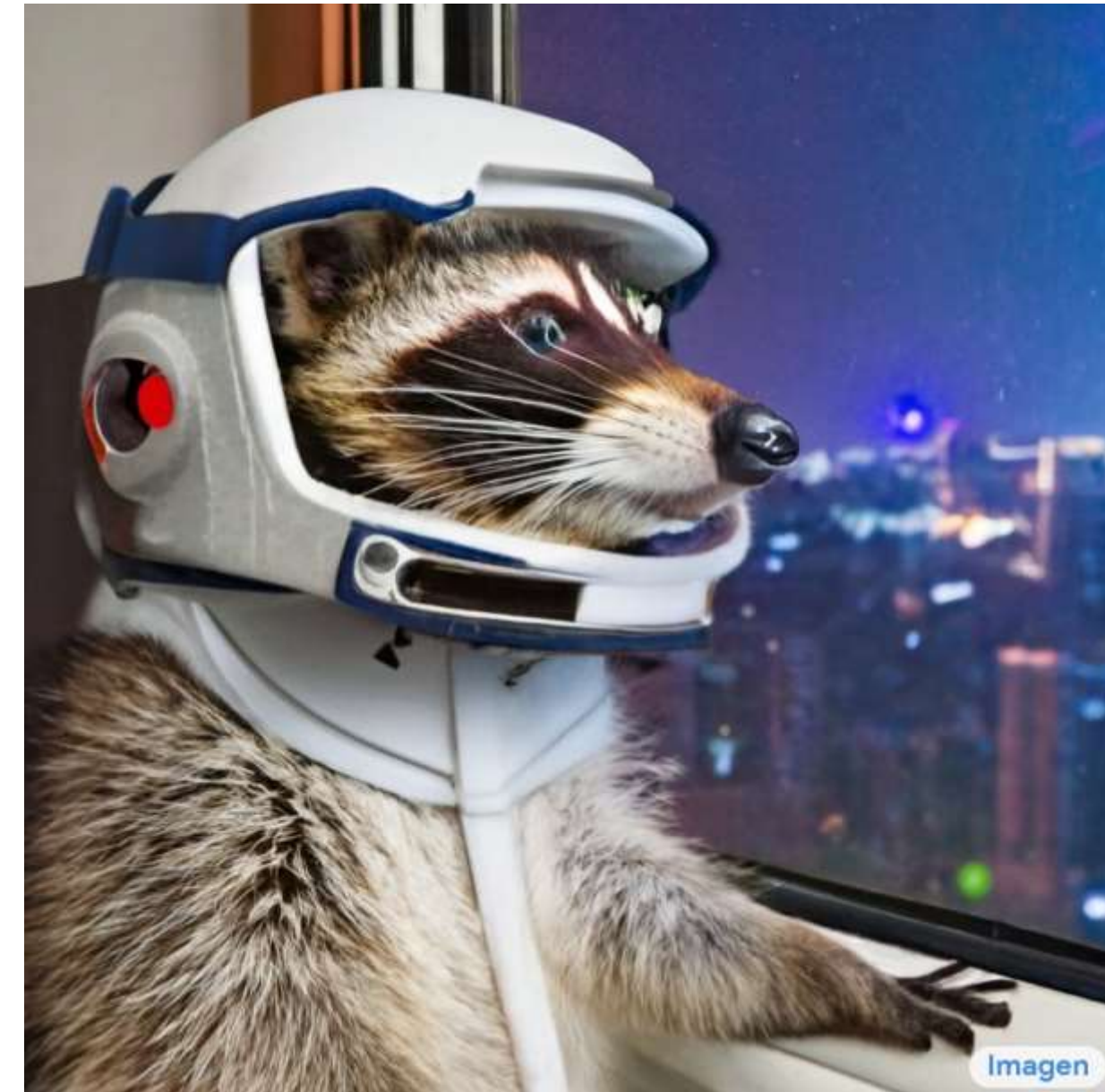
DALL·E 2

“a propaganda poster depicting a cat dressed as french emperor napoleon holding a piece of cheese”



IMAGEN

“A photo of a raccoon wearing an astronaut helmet, looking out of the window at night.”



Impressive conditional diffusion models

Panorama generation

← Generated

Input

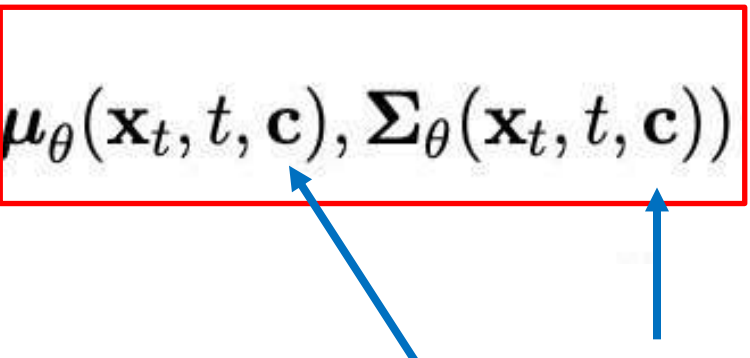
Generated →



Conditional diffusion models

Include condition as input to reverse process

Reverse process: $p_{\theta}(\mathbf{x}_{0:T}|\mathbf{c}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{c}), \quad p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{c}) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t, \mathbf{c}), \Sigma_{\theta}(\mathbf{x}_t, t, \mathbf{c}))$



Condition c injected to model θ

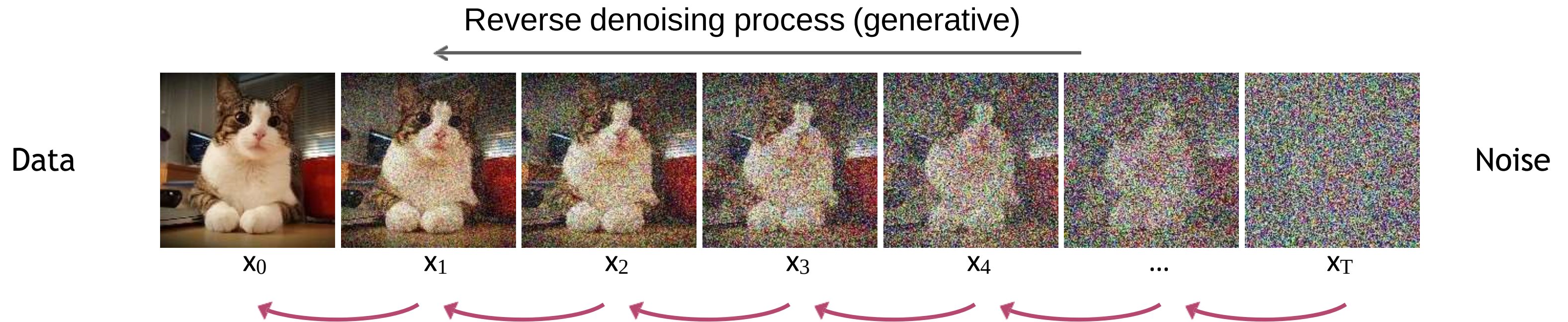
The diagram shows a red rectangular box around the parameters $\mu_{\theta}(\mathbf{x}_t, t, \mathbf{c})$ and $\Sigma_{\theta}(\mathbf{x}_t, t, \mathbf{c})$ in the equation. Two blue arrows point from the text 'Condition c injected to model theta' to the $\mathbf{c}$ argument in both the mean and covariance functions within the box.

Incorporate conditions into U-Net

- Scalar conditioning: encode scalar as a vector embedding, simple spatial addition or adaptive group normalization layers.
- Image conditioning: channel-wise concatenation of the conditional image.
- Text conditioning: single vector embedding - spatial addition or adaptive group norm / a seq of vector embeddings - cross-attention.

Classifier guidance

Using the gradient of a trained classifier as guidance



Denoising process can also be thought as:

- Using denoising U-Net to predict gradient
- Predicted noise as gradient to update $x_t \rightarrow x_{t-1}$
- Direction to clean image $\nabla_{x_t} \log(q(x_t|x_0))$ (the predicted “score function”)

Classifier guidance

Using the gradient of a trained classifier as guidance

Applying Bayes rule to obtain conditional distribution (condition y and image x)

$$p(x | y) = \frac{p(y | x) \cdot p(x)}{p(y)}$$

$$\implies \log p(x | y) = \log p(y | x) + \log p(x) - \log p(y)$$

$$\implies \nabla_x \log p(x | y) = \nabla_x \log p(y | x) + \nabla_x \log p(x),$$

Denoising net prediction

$$\nabla_x \log p_\gamma(x | y) = \nabla_x \log p(x) + \gamma \nabla_x \log p(y | x). \quad \leftarrow \text{Classifier gradient}$$

Guidance scale: value >1 amplifies the influence of classifier signal.

$$p_\gamma(x | y) \propto p(x) \cdot p(y | x)^\gamma.$$

Classifier guidance

Using the gradient of a trained classifier as guidance

$$\nabla_x \log p_\gamma(x | y) = \nabla_x \log p(x) + \gamma \nabla_x \log p(y | x).$$



Samples from an unconditional diffusion model with classifier guidance, for guidance scales 1.0 (left) and 10.0 (right), taken from Dhariwal & Nichol (2021).

Classifier guidance

Using the gradient of a trained classifier as guidance

Algorithm 1 Classifier guided diffusion sampling, given a diffusion model $(\mu_\theta(x_t), \Sigma_\theta(x_t))$, classifier $p_\phi(y|x_t)$, and gradient scale s .

Input: class label y , gradient scale s

$x_T \leftarrow$ sample from $\mathcal{N}(0, \mathbf{I})$

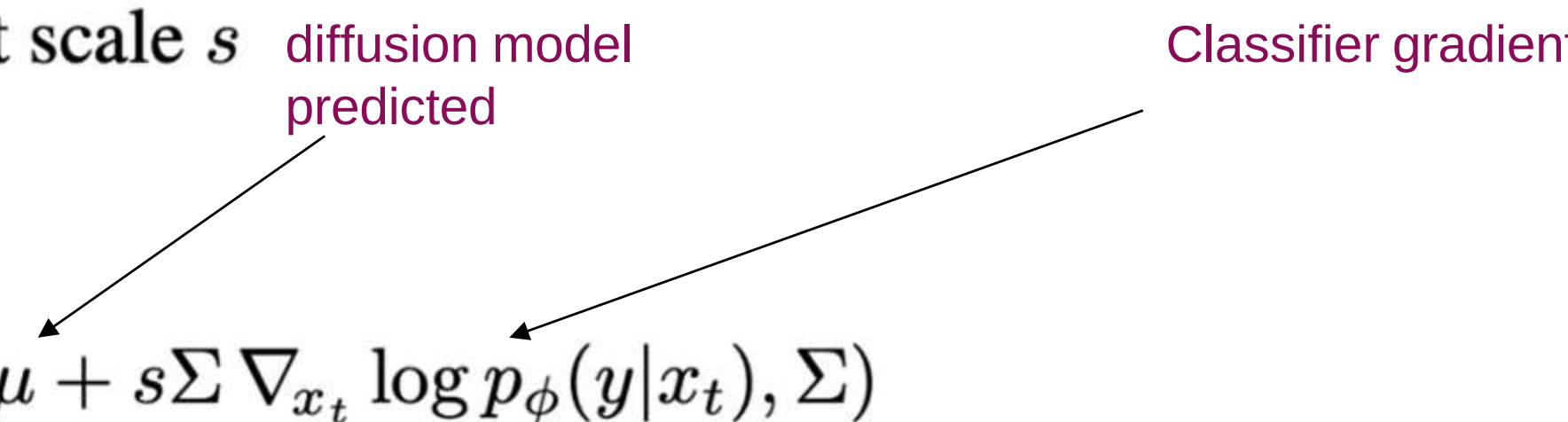
for all t from T to 1 **do**

$\mu, \Sigma \leftarrow \mu_\theta(x_t), \Sigma_\theta(x_t)$

$x_{t-1} \leftarrow$ sample from $\mathcal{N}(\mu + s\Sigma \nabla_{x_t} \log p_\phi(y|x_t), \Sigma)$

end for

return x_0



- Train unconditional Diffusion model
- Take your favorite classifier, depending on the conditioning type
- During inference / sampling mix the gradients of the classifier with the predicted score function of the unconditional diffusion model.

Classifier guidance

Problems of classifier guidance

$$\nabla_x \log p_\gamma(x | y) = \nabla_x \log p(x) + \gamma \nabla_x \log p(y | x). \quad \leftarrow \text{Classifier}$$

Guidance scale: value >1 amplifies the influence of classifier signal.

- At each step of denoising the input to the classifier is a noisy image x_t .
- Classifier is never trained on noisy image. So one needs to re-train classifier on noisy images!
- Most of the information in the input x is not relevant to predicting y .
- Taking the gradient of the classifier w.r.t. its input can yield arbitrary (and even adversarial) directions in input space.

Classifier-free guidance

Get guidance by Bayes' rule on conditional diffusion models

$$p(y | x) = \frac{p(x | y) \cdot p(y)}{p(x)}$$

$$\implies \log p(y | x) = \log p(x | y) + \log p(y) - \log p(x)$$

$$\implies \nabla_x \log p(y | x) = \nabla_x \log p(x | y) - \nabla_x \log p(x).$$

We proved this in
classifier guidance.

$$\nabla_x \log p_\gamma(x | y) = \nabla_x \log p(x) + \gamma \nabla_x \log p(y | x).$$

$$\nabla_x \log p_\gamma(x | y) = \nabla_x \log p(x) + \gamma (\nabla_x \log p(x | y) - \nabla_x \log p(x)),$$

$$\nabla_x \log p_\gamma(x | y) = (1 - \gamma) \nabla_x \log p(x) + \gamma \nabla_x \log p(x | y).$$

Prediction of
unconditional diffusion
model (null condition)

Prediction of
conditional
diffusion model

Classifier-free guidance

Get guidance by Bayes' rule on conditional diffusion models

$$\nabla_x \log p_\gamma(x | y) = (1 - \gamma) \nabla_x \log p(x) + \gamma \nabla_x \log p(x | y).$$

$\gamma = 0$ unconditional generation.

$\gamma = 1$ conditional generation.

$\gamma > 1$ better guidance and better generation quality (empirical)

↑
Score function for
unconditional
diffusion model

↑
Score function for
conditional
diffusion model



Two sets of samples from OpenAI's GLIDE model, for the prompt 'A stained glass window of a panda eating bamboo.', taken from their paper. Guidance scale 1 (no guidance) on the left, guidance scale 3 on the right.

Classifier-free guidance

Trade-off for sample quality and sample diversity



Non-guidance



Guidance scale = 1



Guidance scale = 3

Large guidance weight (ω) usually leads to better individual sample quality but less sample diversity.

Classifier guidance

$$\nabla_x \log p_\gamma(x | y) = \nabla_x \log p(x) + \gamma \nabla_x \log p(y | x).$$

Guidance scale Classifier



X Need to train a separate "noise-robust" classifier
+ unconditional diffusion model.

X Gradient of the classifier w.r.t. input yields
arbitrary values.

Rather than constructing a generative model from classifier, we construct a classifier from a generative model!

Classifier-free guidance

$$\nabla_x \log p_\gamma(x | y) = (1 - \gamma) \nabla_x \log p(x) + \gamma \nabla_x \log p(x | y).$$

Score function for unconditional diffusion model Score function for conditional diffusion model



+ Train conditional & unconditional diffusion
model jointly via drop-out.

+ All pixels in input receive equally 'good'
gradients.

Most recent papers use classifier-free guidance! Very simple yet very powerful idea!

Applications (1): Image Synthesis, Controllable Generation, Text-to-Image



ControlNet

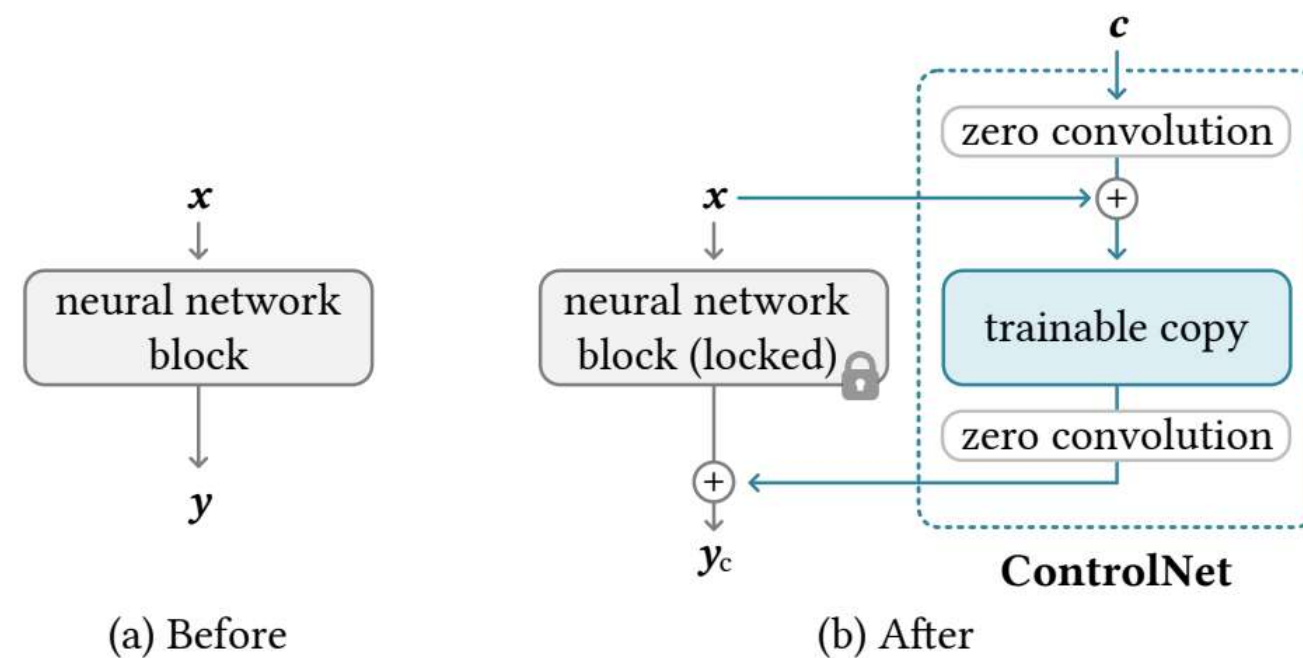
Controlling image diffusion models by conditioning the model with an additional input image



Figure 1: Controlling Stable Diffusion with learned conditions. ControlNet allows users to add conditions like Canny edges (top), human pose (bottom), *etc.*, to control the image generation of large pretrained diffusion models. The default results use the prompt "a high-quality, detailed, and professional image". Users can optionally give prompts like the "chef in kitchen".

ControlNet

Controlling image diffusion models by conditioning the model with an additional input image

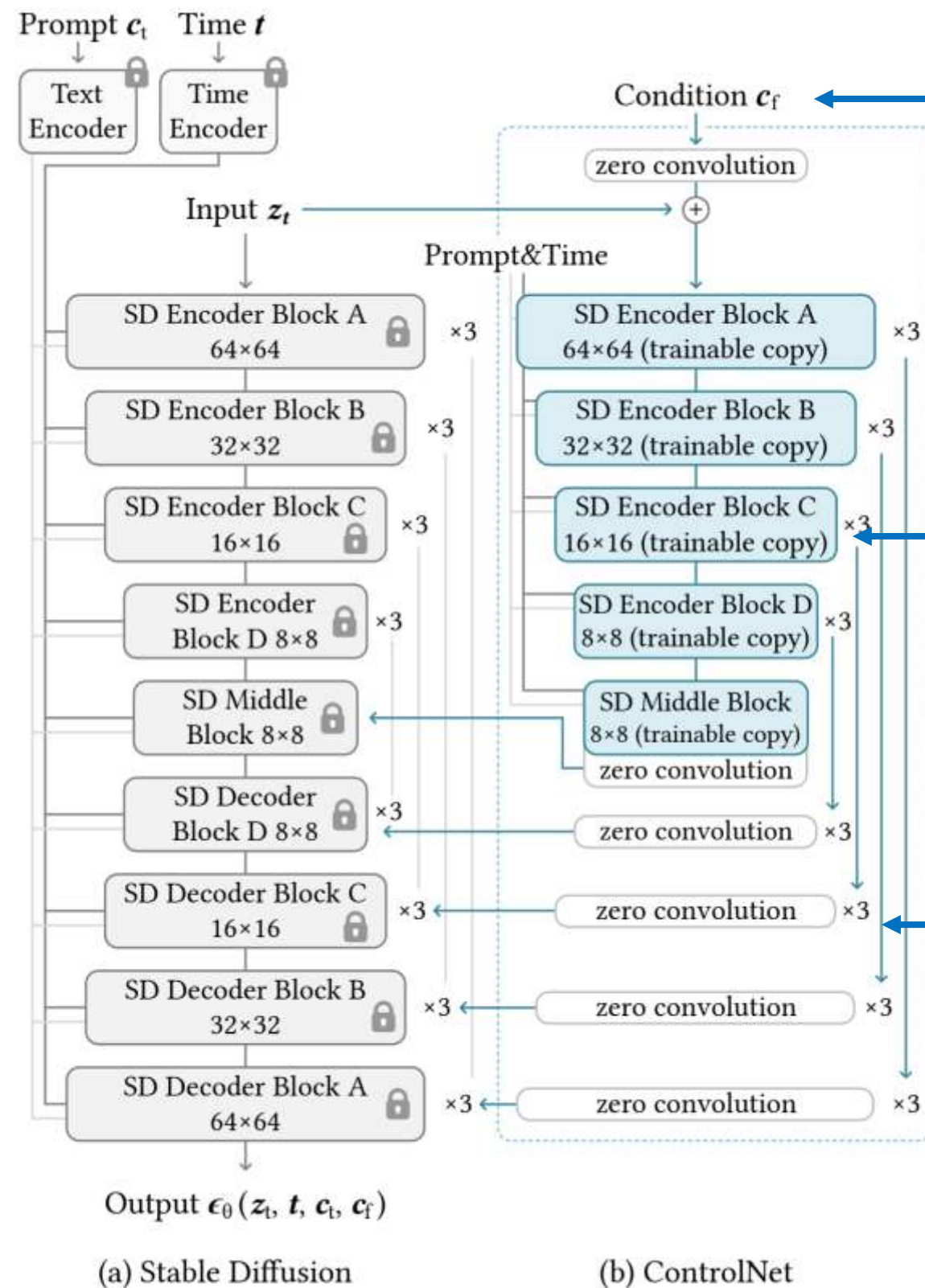


To add a ControlNet to such a block we lock the original block and create a trainable copy and connect them together using zero convolution layers, i.e., 1×1 convolution with both weight and bias initialized to zero.

A ControlNet model has two sets of weights (or blocks) connected by a zero-convolution layer:

- a locked copy keeps everything a large pretrained diffusion model has learned
- a trainable copy is trained on the additional conditioning input

ControlNet



a tiny network $\mathcal{E}(\cdot)$ of four convolution layers (trained jointly with the full model) to encode an image-space condition c_i into a feature space conditioning vector c_f

$$c_f = \mathcal{E}(c_i)$$

ControlNet create a trainable copy of the 12 encoding blocks and 1 middle block of Stable Diffusion. The 12 encoding blocks are in 4 resolutions (64×64 , 32×32 , 16×16 , 8×8) with each one replicated 3 times.

The outputs are added to the 12 skip-connections and 1 middle block of the U-net.

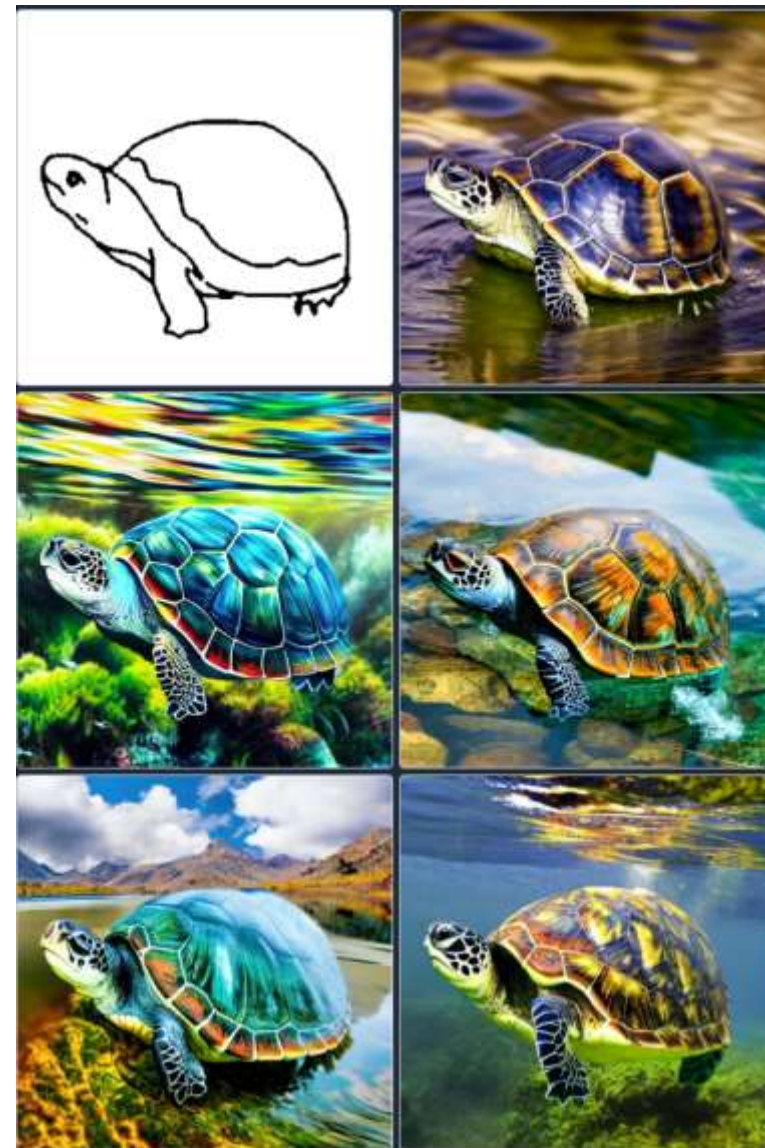
ControlNet

Controlling image diffusion models by conditioning the model with an additional input image

Canny Edge



User Scribbles



Semantic Segmentation



Depth Map



GLIDE

OpenAI

- A 64x64 base model + a 64x64 $\rightarrow$ 256x256 super-resolution model.



“a hedgehog using a calculator”



“a corgi wearing a red bowtie and a purple party hat”



“robots meditating in a vipassana retreat”



“a fall landscape with a small cottage next to a lake”

Samples generated with classifier-free guidance (256x256)

GLIDE

OpenAI

- Fine-tune the model especially for inpainting: feed randomly occluded images with an additional mask channel as the input.



“an old car in a snowy forest”



“a man wearing a white hat”

Text-conditional image inpainting examples

DALL·E 2

OpenAI



a shiba inu wearing a beret and black turtleneck



a close up of a handpalm with leaves growing from it

1kx1k Text-to-image generation.

DALL·E 2

Text Diffs



a photo of a cat → an anime drawing of a super saiyan cat, artstation



a photo of a victorian house → a photo of a modern house



a photo of an adult lion → a photo of lion cub

Change the image CLIP embedding towards the difference of the text CLIP embeddings of two prompts.

Decoder latent is kept as a constant.

Imagen

Google Research, Brain team



A photo of a Shiba Inu dog with a backpack riding a bike. It is wearing sunglasses and a beach hat.



A dragon fruit wearing karate belt in the snow.



A relaxed garlic with a blindfold reading a newspaper while floating in a pool of tomato soup.

Stable Diffusion

Latest & Publicly available text-to-image generation

High-Resolution Image Synthesis with Latent Diffusion Models

Robin Rombach*, Andreas Blattmann*, Dominik Lorenz, Patrick Esser, Björn Ommer

[CVPR '22 Oral](#) | [GitHub](#) | [arXiv](#) | [Project page](#)



[Stable Diffusion](#) is a latent text-to-image diffusion model. Thanks to a generous compute donation from [Stability AI](#) and support from [LAION](#), we were able to train a Latent Diffusion Model on 512x512 images from a subset of the [LAION-5B](#) database. Similar to Google's [Imagen](#), this model uses a frozen CLIP ViT-L/14 text encoder to condition the model on text prompts. With its 860M UNet and 123M text encoder, the model is relatively lightweight and runs on a GPU with at least 10GB VRAM. See [this section](#) below and the [model card](#).

Super-Resolution

Super-Resolution via Repeated Refinement (SR3)

Natural Image Super-Resolution $64 \times 64 \rightarrow 256 \times 256$



Image-to-Image Translation

Palette: Image-to-Image Diffusion Models

Many image-to-image translation applications can be considered as training $p(x|y)$ where y is the input image

For example, for colorization, x is a colored image and y is a gray-level image.

Train a score model for x conditioned on y using:

$$\mathbb{E}_{\mathbf{x}, \mathbf{y}} \mathbb{E}_{\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \mathbb{E}_t ||\epsilon_{\theta}(\mathbf{x}_t, t; \mathbf{y}) - \epsilon||_p^p$$

The conditional score is simply a U-Net with x_t and y concatenated.

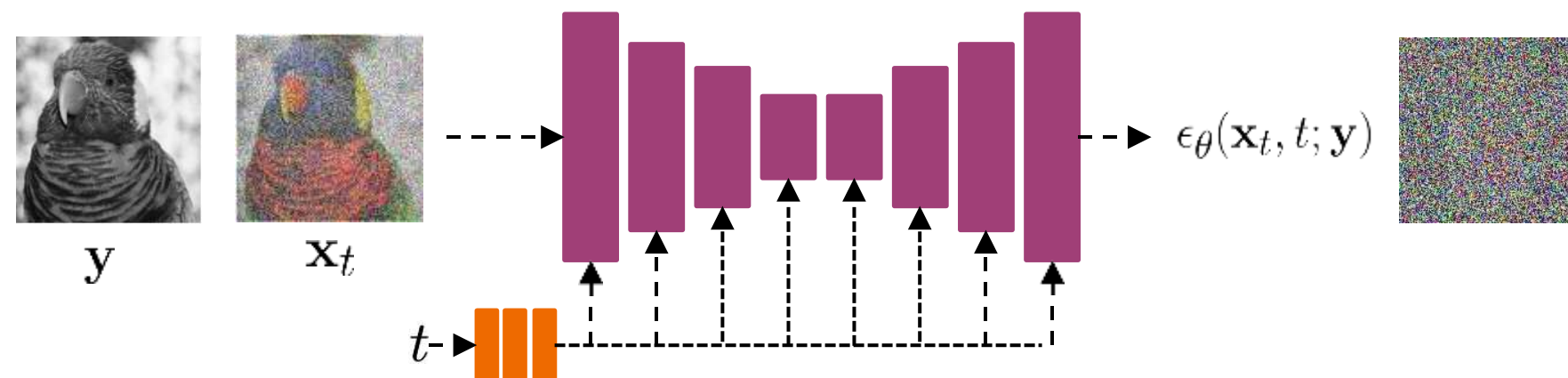
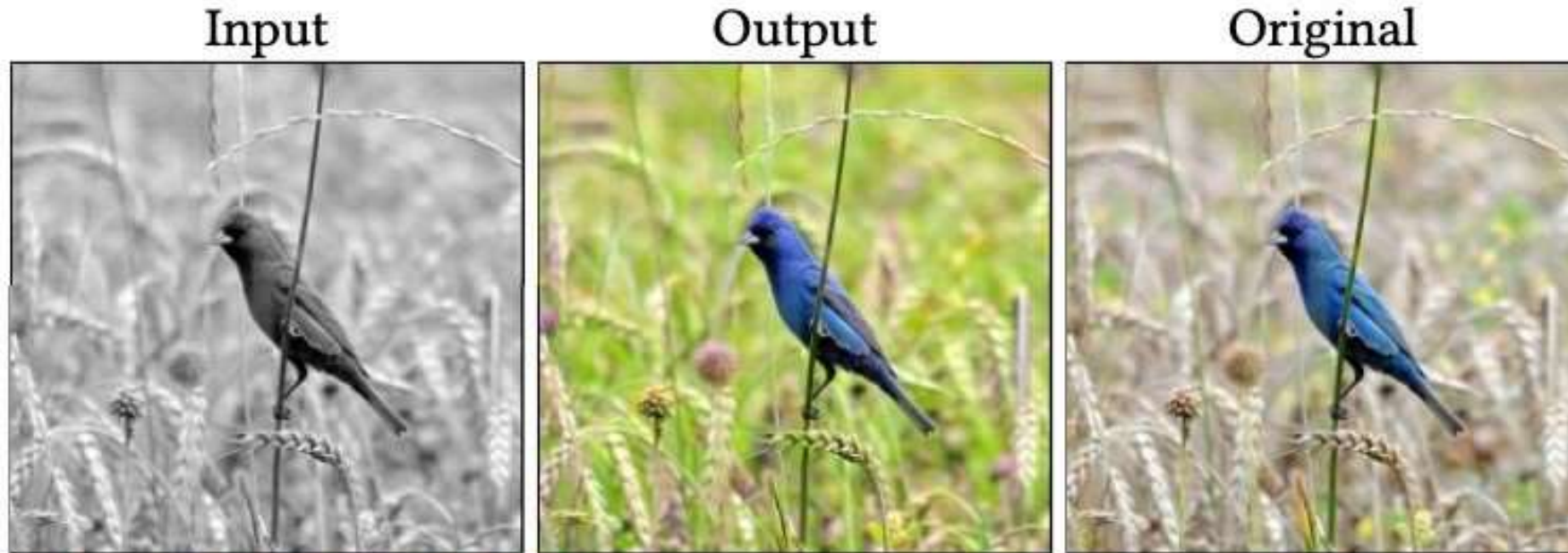


Image-to-Image Translation

Palette: Image-to-Image Diffusion Models

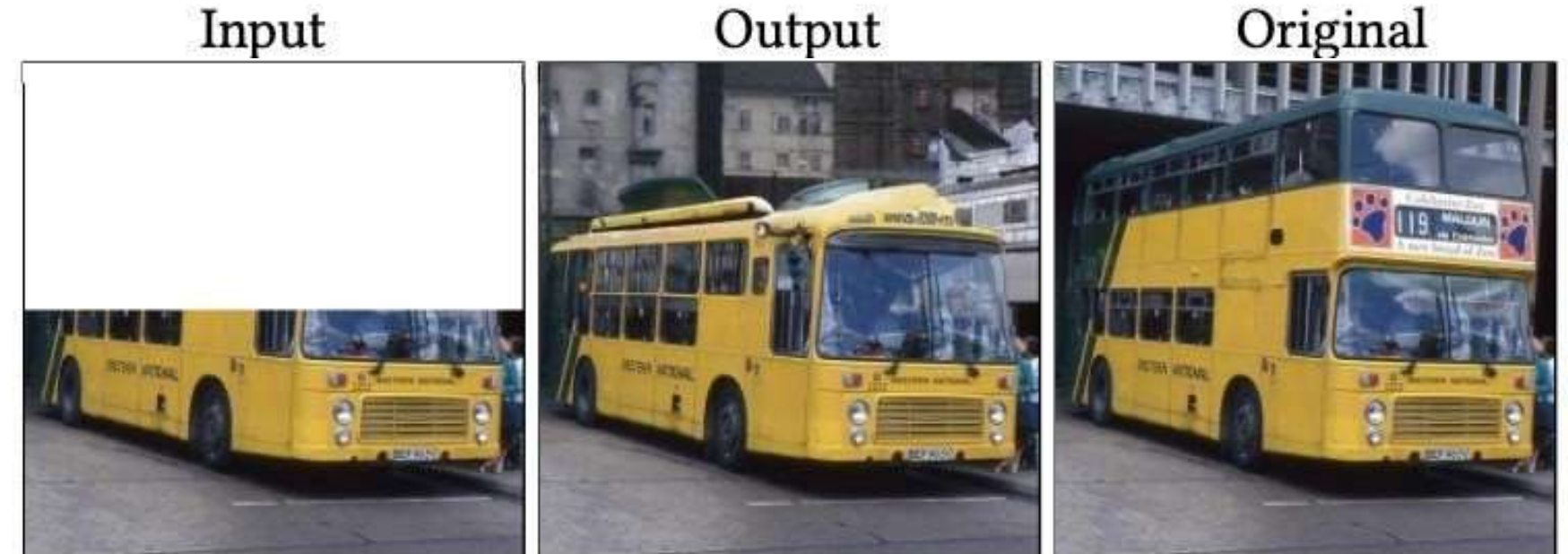
Colorization



Inpainting



Uncropping



JPEG restoration



Video Generation



Samples from a text-conditioned video diffusion model, conditioned on the string ***fireworks***.

[Ho et al., “Video Diffusion Models”, *arXiv*, 2022](#)

[Harvey et al., “Flexible Diffusion Modeling of Long Videos”, *arXiv*, 2022](#)

[Yang et al., “Diffusion Probabilistic Modeling for Video Generation”, *arXiv*, 2022](#)

[Höppe et al., “Diffusion Models for Video Prediction and Infilling”, *arXiv*, 2022](#)

[Voleti et al., “MCVD: Masked Conditional Video Diffusion for Prediction, Generation, and Interpolation”, *arXiv*, 2022](#)

Video Generation

Video Generation Tasks:

- Unconditional Generation (Generate all frames)
- Future Prediction (Generate future from past frames)
- Past Prediction (Generate past from future frames)
- Interpolation (Generate intermediate frames)

➡ Learn a model of the form:

$$p_{\theta}(\mathbf{x}^{t_1}, \dots, \mathbf{x}^{t_K} | \mathbf{x}^{\tau_1}, \dots, \mathbf{x}^{\tau_M})$$

Given frames: $\mathbf{x}^{\tau_1}, \dots, \mathbf{x}^{\tau_M}$

Frames to be predicted: $\mathbf{x}^{t_1}, \dots, \mathbf{x}^{t_K}$

[Ho et al., “Video Diffusion Models”, *arXiv*, 2022](#)

[Harvey et al., “Flexible Diffusion Modeling of Long Videos”, *arXiv*, 2022](#)

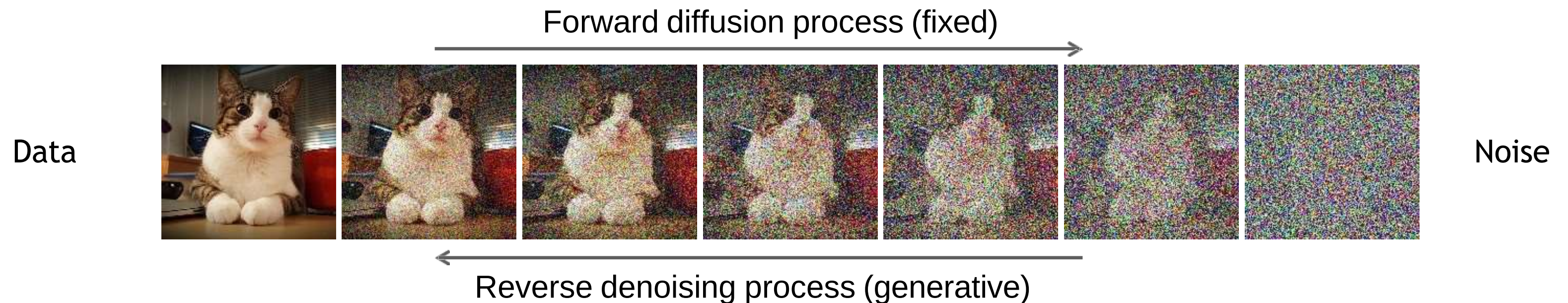
[Yang et al., “Diffusion Probabilistic Modeling for Video Generation”, *arXiv*, 2022](#)

[Höppe et al., “Diffusion Models for Video Prediction and Infilling”, *arXiv*, 2022](#)

[Voleti et al., “MCVD: Masked Conditional Video Diffusion for Prediction, Generation, and Interpolation”, *arXiv*, 2022](#)

Takeaway: Denoising Diffusion Probabilistic Models

We started with denoising diffusion probabilistic models:



We showed how the denoising model can be trained by predicting noise injected in each diffused image:

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1, T)} [||\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)||^2]$$